Jiran: An Evidence-Aware Decision Support Architecture for Resource Allocation and Inventory Redistribution

Asmaa Fatima, Dania Nadeem, Hamdan Khan, Muhammad Abdul Mohsin, Yaasir Bin Muneer, and Youssef Sami Mansi

Abstract Organizations operating multiple locations often hold fragmented inventory and sales data, making it difficult to identify shortages, excess stock, and useful transfers. Jiran is a web-based decision-support architecture that consolidates operational records, presents inventory and exchange workflows, and is designed to support machine-learning-assisted recommendations. This paper reframes the original graduation project as a general resource-allocation system while preserving its implemented inventory domain. The architecture separates ingestion, operational storage, deterministic workflows, and optional intelligence so that the application can remain useful when models or historical data are unavailable. We report a recovery case study in which an expired hosted backend and incompatible model artifacts were replaced by a self-hostable data plane, source-neutral import adapters, and an explicit null state for unvalidated AI output. The current contribution is therefore architectural and methodological rather than an accuracy claim. A validation protocol based on provenance, temporal holdouts, location-level generalization, calibration, and human review is specified for the next experimental phase. This evidence-aware behavior supports trustworthy deployment by refusing to present unsupported recommendations as facts.

1 Introduction

Resource-allocation decisions are often constrained less by the absence of data than by fragmentation across locations, point-of-sale tools, spreadsheets, and operational roles. In an inventory setting, one site can experience a shortage while another holds excess stock. The resulting decision problem requires a

University of Wollongong in Dubai, Dubai, United Arab Emirates

consistent resource snapshot, an auditable transfer workflow, and a defensible method for ranking candidate actions.

Jiran originated as a University of Wollongong in Dubai graduation project for AI-assisted inventory optimization and inter-retailer exchange. Its implemented system combines a React interface, a Supabase-backed operational data layer, optional point-of-sale ingestion, and a Python machine-learning service. The recovery examined in this paper showed that hosted services, credentials, and serialized model artifacts can fail independently of the user-facing application. This motivates a broader and more trustworthy interpretation of the project: an evidence-aware decision-support platform in which intelligence is optional and its unavailability is represented explicitly.

This paper makes four contributions:

1. a generalized architecture for multi-location resource visibility and redistribution while retaining the original inventory use case;
2. a source-neutral ingestion boundary that can accept manual files and exports from open-source enterprise systems;
3. a null-by-design AI contract that prevents stale or incompatible models from producing misleading decisions; and
4. an evaluation protocol that separates system operability, recommendation quality, and business outcomes.

2 Background and Related Work

Inventory distortion and delayed information sharing can amplify variability across supply chains, a phenomenon commonly described as the bullwhip effect [1]. Forecasting in retail is also affected by intermittency, promotions, sparse histories, and differences between aggregate and item-location behavior [2, 3]. These properties make a random row split an inadequate basis for deployment claims: future periods and unseen locations must be evaluated separately.

Jiran also belongs to the class of decision-support systems rather than autonomous control systems. This distinction matters because a useful interface can expose current records and deterministic workflows even when no validated prediction is available. The NIST AI Risk Management Framework emphasizes validity, reliability, transparency, and lifecycle governance [4]. Jiran operationalizes these ideas through source provenance, explicit model status, human confirmation, and abstention when evidence is insufficient.

Table 1 Principal system components and recovery state

Component	Responsibility	Verified recovery state
React/Vite client	User workflows and operational views	Production build succeeds; lint debt remains
Self-hosted Supabase	Authentication, PostgreSQL API, files	Schema migration prepared; local container startup requires completion
Import adapter	Normalize ERP exports to Jiran contract	ERPNext and Odoo modes implemented; unit tests pass
FastAPI service	Prediction and recommendation API	Starts in explicit null-AI mode; health status reports unavailability
ML artifacts	Classification and forecasts	Legacy artifacts rejected as incompatible and unvalidated

3 Problem Definition and Scope

Let L be a set of locations and R a set of resources. At time t, the platform receives an observed quantity $q_{l,r,t}$ and optional demand evidence $d_{l,r,[t-h,t]}$. The operational problem is to identify candidate redistributions from locations with usable excess to locations with a substantiated need, subject to availability, user authorization, and transfer constraints.

The current implementation specializes resources as retail products and locations as retailers or stores. Generalization is limited to terminology and interfaces; it does not claim that the present model has been validated in unrelated domains. The scope includes account management, inventory ingestion, status visibility, exchange requests, operational analytics, and optional recommendation generation. Payments, autonomous ordering, and autonomous transfer execution are outside the current scope.

4 System Architecture

Jiran uses four layers. The presentation layer is a React/Vite web application. The operational layer uses Supabase-compatible authentication, PostgreSQL records, storage, and row-oriented APIs. The integration layer accepts manual CSV data and can normalize exports from ERPNext or Odoo; the original Square/Apideck connector remains optional. The intelligence layer is a FastAPI service designed to host classification, forecasting, and recommendation logic.

The operational data model separates profiles, point-of-sale connections, products, variations, inventory, orders, exchange listings, exchange requests, and analytics snapshots. This separation supports traceability from a displayed resource quantity to its source connection and location.

4.1 Open and Portable Data Plane

The original browser code was tightly coupled to Supabase's JavaScript API. Replacing it with a different backend would require broad changes to authentication, storage, database queries, and edge functions. The recovery therefore preserves the API contract by running the open-source Supabase stack locally. This is a portability intervention, not a change to the project's research objective.

The original Square connector was mediated through Apideck and depended on hosted credentials. Because the project already supported manual ingestion, recovery uses a source-neutral import contract. A tested command-line adapter maps ERPNext item/bin exports and Odoo product exports into the fields required by Jiran. No AI values are generated during normalization.

4.2 Evidence-Aware Intelligence Boundary

The current API exposes a health state with `ai_available=false`, `ai_status=null`, and a machine-readable reason. Prediction, recommendation, and feature-importance endpoints return HTTP 503 while the pipeline is unvalidated. The UI should render this state as "Unavailable—awaiting validated data," not as zero predictions. This avoids three failure modes: presenting stale serialized models as current, interpreting missing data as zero activity, and transforming rule-generated labels into apparently independent model evidence.

5 Operational Workflows and Requirements

Jiran distinguishes retailer, wholesaler, and administrator responsibilities. A retailer maintains locations and inventory, observes shortage or excess indicators, publishes exchange listings, and accepts or rejects proposed transfers. A wholesaler exposes products and responds to restock requests. Administrators supervise account and platform state. These roles are inherited from the graduation project; future deployments should rename them only when a validated domain model requires it.

The minimum deterministic workflow begins with an authenticated operator and a timestamped resource snapshot. Each imported row must identify a source system, location, resource identifier, observed quantity, and observation time. The platform then validates identity and numeric fields, preserves source lineage, and updates the operational view. Exchange candidates are only actionable after users confirm quantities and parties. This workflow remains valid without a trained model.

Functional requirements are grouped as follows. *Data acquisition* requires repeatable import, source identification, schema validation, and duplicate handling. *Operational visibility* requires searchable quantities, locations, statuses, and freshness indicators. *Coordination* requires listings, requests, accept/reject transitions, and transfer records. *Intelligence* requires model versioning, uncertainty, explanation, and an abstention state. *Administration* requires role checks, audit records, and configuration that does not expose credentials to the browser or repository.

Non-functional requirements are equally important. The system should be recoverable from source-controlled schema migrations; should operate locally without a paid hosted service; should avoid transmitting data when connectors are disabled; and should make missingness visible. A stale record must not appear equivalent to a recent zero quantity. Security-sensitive values must be provided at runtime and rotated if version-control exposure occurs.

6 Data Contract and Governance

A trustworthy resource-allocation system needs a canonical data contract before it needs a complex model. For inventory, the minimum normalized fields are a stable resource identifier, name, category, quantity, price, currency, and location. The adapter included in the recovery maps corresponding ERPNext and Odoo export fields into this contract. Rows lacking a resource name are rejected; invalid numbers are normalized conservatively; no prediction, confidence, or recommendation field is created by the adapter.

For an ML experiment, the canonical contract expands to include event time, source system, connection identifier, unit semantics, transaction direction, and any adjustment reason. Snapshot and transaction tables must not be silently mixed. Every dataset release should have a manifest that records:

- source system and extraction method;
- inclusive time range and timezone;
- participating locations and resource coverage;
- raw and retained row counts, duplicates, missing values, and rejected rows;
- schema and adapter versions; and
- cryptographic checksums of immutable input files.

Access control should follow least privilege. Browser clients should use public client credentials and row-level policies, while privileged server keys should be confined to server-side components. The recovered repository previously contained environment files and hard-coded tokens; removing them from the latest tree does not remove them from Git history. Key rotation and, if the repository remains public, history rewriting are therefore required. Operational and research datasets should exclude secrets and unnecessary personal data.

Data retention and consent depend on the deployment. Jiran's current records primarily describe products, quantities, business profiles, and transactions, but account details may still identify people. A deployment must define purpose, retention, deletion, export, and incident procedures before collecting new production data. Research exports should use pseudonymous location identifiers unless identity is analytically necessary.

7 Trustworthy-AI Controls

ISDIA-2027 emphasizes trustworthy AI, governance, risk, and responsible innovation. Jiran addresses this theme through controls at the data, model, interface, and operational levels.

7.1 Data-Level Controls

Schema validation, provenance manifests, freshness checks, and coverage thresholds prevent malformed or partial snapshots from entering evaluation unnoticed. Unknown categories are counted rather than silently mapped to a frequent class. Missing history triggers an unavailable state rather than an assumed demand of zero.

7.2 Model-Level Controls

Model training must avoid target leakage. The legacy training code generated labels from rule thresholds and also supplied closely related rule inputs as features; this makes an apparently accurate model capable of reproducing its label generator without demonstrating future decision value. The revised protocol will separate any heuristic baseline from supervised targets and document how ground truth is created. Serialized artifacts must be tied to exact code, dependency versions, training-manifest checksums, and evaluation reports.

Temporal evaluation is mandatory because random splitting can place observations from the same item and period family on both sides of the split. A second holdout by location tests generalization. Calibration curves and abstention thresholds should accompany accuracy metrics. Recommendations that violate quantity, authorization, or transfer constraints are rejected regardless of model score.

7.3 Interface and Human Oversight

The interface should show data recency, source, model state, confidence, and a concise reason for each recommendation. Operators retain authority over exchange and transfer actions. Rejections and modifications become auditable feedback but should not be treated automatically as training labels: an operator's decision may reflect transport cost, policy, or information absent from the model.

7.4 Operational Monitoring

Post-deployment monitoring should compare live input distributions with the validated dataset, track abstention and override rates, and alert on stale ingestion. Changes to adapters, schemas, thresholds, or models require a new version and regression checks. Rollback must restore both code and its compatible data/model contract.

8 Research Questions and Experimental Design

The next empirical study is organized around four research questions.

RQ1: Does the platform improve operators' ability to identify and complete feasible redistributions compared with existing spreadsheet or isolated-POS workflows? This will be measured with task success, time, error count, and usability responses in a counterbalanced study.

RQ2: Do forecasting or classification models improve later-period shortage/excess detection over transparent baselines? Baselines will include last-observation, seasonal naive, simple moving average, and documented inventory thresholds. Statistical comparisons should report uncertainty across products and locations, not only a pooled point estimate.

RQ3: Do recommendation constraints reduce shortage and excess without generating infeasible transfers? Offline replay will preserve chronology and compare service level, holding proxy, shortage duration, transfer count, and constraint violations. Where cost data are absent, the paper will not infer monetary savings.

RQ4: Can users recognize AI unavailability and uncertainty and make an informed decision? A human-factors test will compare null, low-confidence, and high-confidence states, measuring comprehension and inappropriate reliance.

The study population should include users familiar with inventory or operational allocation. Sample size will be justified after selecting the primary outcome and effect-size assumption. The protocol should distinguish exploratory

metrics from confirmatory outcomes and document exclusions before analysis. If only synthetic data are available, results will be labeled simulation evidence and separated from claims about real operations.

9 Generalization Boundary

The architecture can represent resources beyond grocery products because its core entities are source, location, resource, quantity, event, and transfer. Potential domains include spare parts, equipment, supplies, and capacity units. However, semantics differ: resources may expire, be indivisible, require certification, or have transportation and compatibility constraints. Generalization is therefore an architectural hypothesis, not a demonstrated outcome. This paper preserves inventory as the evaluated domain and treats any broader deployment as a new validation setting.

10 Recovery Method

The recovery followed five stages. First, the repository, final report, service contracts, and tracked configuration were inventoried. Second, the legacy hosted Supabase endpoint was tested and found not to resolve; no historical rows were recovered. Third, credentials found in version control were removed from the working tree and local configuration was made environment-based. Fourth, a local schema migration and import boundary were added. Fifth, the ML service was changed to start safely without loading incompatible artifacts.

No historical performance figure was carried into this paper. Repository inspection found that training data needed for reproduction was absent, legacy artifacts could not be loaded under the recovered environment, and generated outputs did not constitute a valid held-out evaluation. These conditions make abstention the only defensible current behavior.

11 Validation Protocol

Future experiments will use an immutable dataset manifest containing source, location coverage, time range, record counts, schema version, missingness, and a checksum. The evaluation should answer four separate questions:

1. **Operability:** Can users authenticate, import resources, view quantities, and complete exchange workflows?

2. **Predictive validity:** Do models outperform transparent baselines on later time periods and held-out locations?
3. **Decision quality:** Are proposed transfers feasible, and do they reduce shortage and excess without producing negative stock?
4. **Human factors:** Can operators understand the evidence, confidence, and consequences before approval?

Metrics should include mean absolute error or scaled error for demand forecasts, precision and recall for shortage/excess alerts, calibration for confidence scores, inventory constraint violations, service-level change, transfer acceptance, and user task completion. Temporal splits must precede any tuning on the test period. Location-level holdouts should measure portability. Every recommendation should retain the data snapshot, model version, rationale, and operator decision.

12 Current Results and Limitations

The current results are engineering verification results, not claims of AI effectiveness. The React production build completes. Two import-adapter unit tests pass. The FastAPI service starts and reports the null AI state; prediction requests receive HTTP 503 rather than fabricated output. A standalone progress dashboard and high-level architecture view pass desktop and mobile overflow checks.

The local Supabase stack starts successfully, applies the recovery migration, and passes an automated authentication/profile/inventory round trip. The frontend still has existing lint violations and a large JavaScript bundle, and the legacy Square/Apideck edge path has not been reactivated. The original database is unreachable, the original training dataset is absent, and the legacy model artifacts are incompatible with the recovered dependency environment. Consequently, forecast accuracy, recommendation quality, and business impact are intentionally unreported.

13 Discussion

The principal lesson is that graceful degradation is part of trustworthy intelligent-system design. Treating the AI module as optional enables the platform to preserve deterministic value while its evidence base is rebuilt. It also creates a clear experimental boundary: once representative data is available, the same interface can expose validated recommendations without changing the operational schema.

This approach preserves continuity with the graduation project. The core problem remains inventory visibility and redistribution; the general resource-

allocation framing only clarifies how the architecture may transfer to adjacent settings. Claims beyond inventory should be tested rather than inferred.

14 Conclusion and Next Steps

Jiran provides a practical architecture for consolidating resource records, exposing shortages and excess, and coordinating redistribution. The recovery replaced mandatory hosted dependencies with a self-hostable data plane and source-neutral import path, while changing the AI subsystem to abstain when evidence is missing or invalid. The next phase will ingest a provenance-controlled dataset, repair the evaluation design, conduct temporal and location holdouts, and run a structured usability study. Until those gates pass, Jiran should be described as an AI-ready decision-support platform, not as a validated AI optimizer.

Acknowledgements

The original Jiran capstone was supervised by Dr. Farhad Oroumchian at the University of Wollongong in Dubai. Authorship, corresponding-author details, and acknowledgements must be confirmed by all contributors before submission.

References

1. Lee, H.L., Padmanabhan, V., Whang, S.: Information distortion in a supply chain: The bullwhip effect. Management Science 43(4), 546–558 (1997). https://doi.org/10.1287/mnsc.43.4.546
2. Syntetos, A.A., Boylan, J.E.: The accuracy of intermittent demand estimates. International Journal of Forecasting 21(2), 303–314 (2005). https://doi.org/10.1016/j.ijforecast.2004.10.001
3. Fildes, R., Ma, S., Kolassa, S.: Retail forecasting: Research and practice. International Journal of Forecasting 35(1), 1–9 (2019). https://doi.org/10.1016/j.ijforecast.2018.06.004
4. Tabassi, E. (ed.): Artificial Intelligence Risk Management Framework (AI RMF 1.0). NIST AI 100-1, National Institute of Standards and Technology (2023). https://doi.org/10.6028/NIST.AI.100-1